

LAB 02 — Language Models

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | Học kỳ: Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ Hạn nộp: 23:59 - 30/09/2026

Chủ đề: N-gram Language Models, Smoothing and Perplexity

1. Bối cảnh

Ở LAB 01, văn bản được biểu diễn thành vector TF-IDF để phục vụ tìm kiếm.

Tuy nhiên, TF-IDF không mô hình hóa trực tiếp **xác suất của một chuỗi từ**.

Ví dụ, xét hai câu:

the cat eats fish

the fish eats cat

TF-IDF có thể biểu diễn cả hai câu bằng các thành phần từ tương tự nhau, nhưng không trực tiếp trả lời:

- Câu nào có khả năng xuất hiện tự nhiên hơn?
- Sau the cat từ nào có khả năng xuất hiện tiếp theo?
- Xác suất của cả câu là bao nhiêu?
- Một câu mới có “giống ngôn ngữ” của corpus hay không?

Language Model giải quyết nhóm vấn đề này bằng cách mô hình hóa:

$$P(w_1, w_2, \dots, w_T).$$

Trong lab này, sinh viên xây dựng một **n-gram language model từ đầu**, sau đó sử dụng nó để dự đoán và đánh giá văn bản.

2. Mục tiêu học tập

Sau khi hoàn thành lab, sinh viên có thể:

- **LO1:** Giải thích language model dưới góc nhìn xác suất.
- **LO2:** Tính xác suất của chuỗi từ bằng chain rule.
- **LO3:** Giải thích giả định Markov của n-gram language model.
- **LO4:** Tự cài đặt unigram, bigram và trigram language model.
- **LO5:** Giải thích vấn đề zero probability.
- **LO6:** Cài đặt và so sánh các phương pháp smoothing cơ bản.
- **LO7:** Tính và diễn giải perplexity.
- **LO8:** Sử dụng language model để dự đoán từ tiếp theo hoặc xếp hạng câu.
- **LO9:** Phân tích ảnh hưởng của corpus, vocabulary và smoothing đến mô hình.
- **LO10:** Nhận diện giới hạn của n-gram language models và liên hệ tới neural language models.

3. Kiến thức tiên quyết

Sinh viên cần hoàn thành LAB 01 và nắm:

- tokenization;
- vocabulary;
- frequency/count;
- xác suất cơ bản;
- vector và normalization ở mức cơ bản;

- Python và NumPy;
- train/test split.

Không yêu cầu sinh viên đã học Transformer.

4. Cấu trúc lab

Giai đoạn	Thời lượng	Hoạt động
Theory recap	20 phút	Language modeling và xác suất
Calculation	30 phút	Chain rule, n-gram, smoothing
Prediction	10 phút	Dự đoán trước khi chạy
Experiment 1	20 phút	Corpus và n-gram statistics
Core implementation	40 phút	Unigram/Bigram/Trigram
Experiment 2	30 phút	Zero probability và smoothing
Evaluation	25 phút	Perplexity
Application	25 phút	Next-word prediction / sentence ranking
Error analysis	20 phút	Phân tích lỗi
Reflection	10 phút	Từ n-gram đến neural LM

5. Theory recap

5.1 Language Model là gì?

Language model gán xác suất cho một chuỗi từ:

$$P(w_1, w_2, \dots, w_T).$$

Ví dụ:

$$P(\text{the cat eats fish}).$$

Một language model tốt thường gán xác suất cao hơn cho những chuỗi có tính tự nhiên cao hơn trong ngôn ngữ/corpus đang xét.

5.2 Chain Rule

Theo chain rule:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1}).$$

Ví dụ:

$$P(\text{the cat eats})$$

được phân rã thành:

$$P(\text{the})P(\text{cat}|\text{the})P(\text{eats}|\text{the cat}).$$

Đây là nền tảng của language modeling.

5.3 Vấn đề của chain rule

Nếu sử dụng toàn bộ lịch sử:

$$P(w_t|w_1, \dots, w_{t-1}),$$

số lượng context có thể tăng cực nhanh.

Do đó n-gram language model sử dụng một giả định đơn giản hóa.

6. N-gram Language Model

6.1 Unigram

Unigram chỉ xét chính từ hiện tại:

$$P(w_t).$$

Ước lượng:

$$P(w) = \frac{C(w)}{N}.$$

Trong đó:

- $C(w)$: số lần từ w xuất hiện;
- N : tổng số token.

6.2 Bigram

Bigram xét một từ trước đó:

$$P(w_t|w_{t-1}).$$

Ước lượng bằng maximum likelihood:

$$P(w_t|w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}.$$

Ví dụ:

```
the cat
the dog
the fish
```

Nếu:

$$C(\text{the cat}) = 20$$

và

$$C(\text{the}) = 100,$$

thì:

$$P(\text{cat}|\text{the}) = 0.2.$$

6.3 Trigram

Trigram sử dụng hai từ trước:

$$P(w_t|w_{t-2}, w_{t-1}).$$

Ước lượng:

$$P(w_t|w_{t-2}, w_{t-1}) = \frac{C(w_{t-2}, w_{t-1}, w_t)}{C(w_{t-2}, w_{t-1})}.$$

7. Bài tập tính toán

Phần này phải được làm **trước khi sử dụng code**.

Bài 1 — Unigram

Cho corpus:

```
the cat eats fish  
the cat likes fish  
the dog eats meat
```

Hãy:

1. Xác định vocabulary.
2. Tính tổng số token.
3. Tính $P(w)$ cho:

```
the  
cat  
fish  
dog
```

4. Kiểm tra:

$$\sum_w P(w) = 1.$$

Bài 2 — Bigram

Với corpus trên, tính:

$$P(\text{cat}|\text{the})$$

$$P(\text{dog}|\text{the})$$

$$P(\text{eats}|\text{cat})$$

$$P(\text{likes}|\text{cat})$$

Sau đó trả lời:

Tại sao tổng xác suất của các từ đứng sau the phải bằng 1 nếu vocabulary và context được xử lý đầy đủ?

Bài 3 — Xác suất câu

Cho:

the cat eats fish

Giả sử sử dụng bigram:

$$P(\text{the})P(\text{cat}|\text{the})P(\text{eats}|\text{cat})P(\text{fish}|\text{eats}).$$

Tính xác suất của câu.

Sau đó trả lời:

Nếu thêm một từ vào câu, xác suất của cả câu có thể tăng không?

Đây là câu hỏi quan trọng để sinh viên nhận ra rằng **xác suất của một sequence không phải là thước đo trực tiếp để so sánh các câu có độ dài khác nhau.**

Bài 4 — Sentence ranking

Cho hai câu:

S1 = the cat eats fish
S2 = the dog eats fish

Dựa trên corpus đã cho, hãy dự đoán câu nào có xác suất cao hơn.

Không chạy code trước khi đưa ra dự đoán.

8. Zero Probability

Xét:

the cat eats fish

Nếu corpus không chứa:

cat eats

thì:

$$C(\text{cat eats}) = 0.$$

Do đó:

$$P(\text{eats}|\text{cat}) = 0.$$

Khi tính xác suất cả câu:

$$P(S) = \prod_t P(w_t|w_{t-1}),$$

chỉ cần một thành phần bằng 0 thì:

$$P(S) = 0.$$

Đây là **zero-frequency problem**.

9. Bài tập suy luận trước khi smoothing

Cho corpus:

```
I like NLP
I like AI
I study NLP
```

Giả sử cần tính:

$$P(\text{AI}|\text{study}).$$

Sinh viên phải trả lời:

1. Count của study AI là bao nhiêu?
2. Xác suất MLE là bao nhiêu?
3. Điều gì xảy ra khi tính xác suất câu chứa bigram này?
4. Điều này có nghĩa mô hình “biết” rằng câu đó chắc chắn không thể xảy ra không?

Mục tiêu là phân biệt:

Không quan sát thấy $\neq$ xác suất thực sự bằng 0.

10. Smoothing

10.1 Add-one / Laplace smoothing

Một cách đơn giản:

$$P_{\text{Laplace}}(w|h) = \frac{C(h, w) + 1}{C(h) + V}.$$

Trong đó:

- (h): context;
- (V): vocabulary size.

Ví dụ:

$$P(w|h) = \frac{C(h, w) + 1}{C(h) + V}.$$

Sinh viên cần chú ý rằng **smoothing làm thay đổi toàn bộ phân phối xác suất**, không chỉ biến những zero thành số dương.

11. Bài tập tính smoothing

Cho:

$$C(\text{cat}) = 10$$

$$C(\text{cat eats}) = 0$$

và:

$$V = 5.$$

Tính:

$$P_{\text{Laplace}}(\text{eats}|\text{cat}).$$

Sau đó tính lại khi:

$$C(\text{cat eats}) = 3.$$

Trả lời:

Smoothing đã thay đổi xác suất của những bigram khác như thế nào?

12. Prediction trước experiment

Trước khi code, sinh viên phải ghi dự đoán.

Prediction 1

Khi chuyển:

unigram → bigram → trigram

vocabulary có tăng không?

Prediction 2

Số lượng n-gram sẽ thay đổi như thế nào?

Prediction 3

Mô hình nào có khả năng gặp zero probability nhiều hơn?

Prediction 4

Mô hình nào dự kiến có perplexity thấp hơn trên **training set**?

Prediction 5

Nếu corpus rất nhỏ, trigram có chắc chắn tốt hơn bigram không?

Sinh viên phải ghi:

Prediction:
Reason:
Confidence:

Sau experiment mới được sửa prediction.

13. Experiment 1 — Corpus statistics

Sử dụng corpus đã chuẩn bị cho môn học.

Sử dụng corpus nhỏ trước:

10K–30K documents

Sinh viên thống kê:

- số documents;
- số sentences;
- số tokens;
- vocabulary size;
- số unigram;
- số bigram;
- số trigram;
- số n-gram chỉ xuất hiện một lần.

Ví dụ:

Model	# unique n-grams
Unigram	...
Bigram	...
Trigram	...

Sau đó vẽ:

Frequency distribution của unigram/bigram/trigram.

14. Core implementation

Đây là phần quan trọng nhất của lab.

Sinh viên **không được dùng trực tiếp** một thư viện language model để thay thế implementation.

Cần tự xây dựng:

```
build_vocabulary()
count_ngrams()
train_unigram()
```

```
train_bigram()
train_trigram()
probability()
sentence_probability()
```

Có thể thiết kế class:

```
class NGramLanguageModel:
    def __init__(self, n):
        ...

    def fit(self, corpus):
        ...

    def probability(self, context, word):
        ...

    def sentence_probability(self, sentence):
        ...

    def next_word_distribution(self, context):
        ...
```

15. Log probability

Không nên tính:

$$P(S) = \prod_t P(w_t | context)$$

trực tiếp với những câu dài.

Thay vào đó:

$$\log P(S) = \sum_t \log P(w_t | context).$$

Sinh viên cần implement:

```
sentence_log_probability()
```

và giải thích:

Tại sao log probability giúp tránh vấn đề numerical underflow?

16. Experiment 2 — MLE vs smoothing

So sánh:

1. MLE;
2. Laplace smoothing.

Đánh giá trên:

- training set;
- validation set;
- test set.

Ví dụ:

Model	Train PPL	Valid PPL	Test PPL
Bigram MLE	...	...	...
Bigram Laplace	...	...	...
Trigram MLE	...	...	...
Trigram Laplace	...	...	...

Không yêu cầu sinh viên chỉ ra “model nào thắng”.

Thay vào đó phải trả lời:

Tại sao kết quả lại như vậy?

17. Perplexity

Perplexity là metric quan trọng của language model.

Với sequence gồm (N) token:

$$PP(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}}.$$

Tương đương:

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | context_i) \right).$$

Ý tưởng trực quan:

Perplexity đo mức độ “bối rối” của model khi quan sát sequence.

Perplexity thấp hơn thường có nghĩa model gán xác suất cao hơn cho dữ liệu đánh giá.

Nhưng sinh viên phải lưu ý:

Perplexity chỉ có ý nghĩa khi các model được đánh giá trên cùng dữ liệu, cùng preprocessing và cùng cách tính.

18. Bài tập tính Perplexity

Cho:

$$P(w_1) = 0.5$$

$$P(w_2 | w_1) = 0.25$$

$$P(w_3 | w_2) = 0.5.$$

Tính:

$$P(W)$$

và:

$$PP(W).$$

Sau đó tính lại nếu:

$$P(w_2|w_1) = 0.1.$$

Trả lời:

Vì sao chỉ một xác suất nhỏ cũng có thể làm perplexity thay đổi đáng kể?

19. Experiment 3 — Perplexity

So sánh:

```
Unigram
Bigram
Trigram
```

trên:

```
train
validation
test
```

Kết quả:

Model	Train	Validation	Test
Unigram			
Bigram			
Trigram			

Sinh viên phải giải thích:

- tại sao training perplexity thay đổi;
- tại sao validation/test có thể không cải thiện;
- dấu hiệu overfitting;
- tác động của corpus size.

20. Application — Next-word prediction

Xây dựng một hệ thống dự đoán từ tiếp theo.

Input:

```
the cat
```

Output:

```
1. eats      0.42
2. likes     0.31
3. sees      0.08
```

4. ...

Sinh viên phải thử ít nhất:

5 contexts

và ghi lại:

Context	Prediction	Actual next word
the cat	...	...
natural language	...	...
machine learning	...	...

21. Application — Sentence ranking

Cho một context và nhiều candidate continuations.

Ví dụ:

Context:
machine learning

A. is useful for NLP
B. banana computer quickly
C. studies language models

Language model tính:

$$P(A|context)$$

$$P(B|context)$$

$$P(C|context).$$

Xếp hạng các candidate.

Đây là một application đơn giản nhưng giúp sinh viên thấy:

Language model không chỉ “sinh text”; nó cũng có thể được dùng để **score và rank text**.

22. Error analysis

Sinh viên chọn:

- 2 prediction đúng;
- 2 prediction sai.

Với mỗi trường hợp:

Context:
Model prediction:
Expected:

Probability:

Sau đó xác định nguyên nhân:

- insufficient training data;
- unseen n-gram;
- vocabulary limitation;
- sparsity;
- context quá ngắn;
- domain mismatch;
- preprocessing;
- smoothing.

23. Một thí nghiệm rất quan trọng: context length

Cho cùng một corpus:

Unigram
Bigram
Trigram

Hỏi:

Context dài hơn có luôn tốt hơn không?

Sinh viên phải dựa trên:

- training perplexity;
- validation perplexity;
- test perplexity;
- số lượng n-gram;
- số lượng unseen n-gram.

Đây là điểm chuyển rất tự nhiên sang:

n-gram
↓
neural language model
↓
word embeddings
↓
RNN
↓
attention
↓
Transformer

24. Reflection

Sinh viên trả lời ngắn:

Câu 1

Nếu tăng (n), mô hình nhận thêm thông tin gì?

Câu 2

Tại sao tăng (n) lại làm sparsity tăng?

Câu 3

Tại sao smoothing cần thiết?

Câu 4

Perplexity đo điều gì?

Câu 5

Một model có perplexity thấp hơn có **luôn** tạo ra văn bản tốt hơn đối với con người không? Giải thích.

Câu 6

N-gram language model thất bại ở đâu khi so với cách con người hiểu ngôn ngữ?

Câu 7

Nếu context dài 100 từ, trigram có sử dụng được thông tin của 97 từ đầu không?

Câu cuối là cầu nối trực tiếp sang neural language models.

25. AI policy

AI được phép sử dụng cho:

- tìm documentation;
- debugging;
- giải thích lỗi Python;
- kiểm tra API;
- visualization;
- tối ưu code sau khi implementation cơ bản đã hoàn thành.

Không được sử dụng AI trong:

- bài tính toán ban đầu;
- prediction trước experiment;
- giải thích kết quả;
- error analysis;
- reflection;
- individual learning check.

Nếu sử dụng AI cho code:

AI assistance statement

Tool:

Purpose:

What was generated:

What was modified:

How the result was verified:

26. Individual learning check

Mỗi sinh viên có khoảng **3 phút**.

Giảng viên hoặc TA chọn ngẫu nhiên 1 câu:

Vì sao bigram có zero probability?

hoặc:

Tại sao phải dùng log probability?

hoặc:

Perplexity thấp nghĩa là gì?

hoặc:

Tại sao trigram không nhất thiết tốt hơn bigram trên test set?

hoặc:

Nếu cat eats chưa xuất hiện trong training corpus thì model xử lý thế nào?

Chấm:

Mức	Điểm
Không giải thích được	0
Hiểu một phần	1
Giải thích chính xác	2

27. Deliverables

```
lab02/
├── README.md
├── calculations.md
├── prediction.md
├── ngram_lm.py
├── experiments.ipynb
├── results.csv
├── error_analysis.md
└── reflection.md
```

Trong đó:

File	Nội dung
calculations.md	Tính toán bằng tay
prediction.md	Dự đoán trước experiment
ngram_lm.py	Implementation
experiments.ipynb	Experiments
results.csv	Perplexity + prediction results
error_analysis.md	Phân tích lỗi
reflection.md	Câu hỏi tổng kết

28. Rubric

Thành phần	Điểm
Theory & calculation	10
Prediction	10

Thành phần	Điểm
Core implementation	20
Smoothing experiment	15
Perplexity evaluation	15
Application	10
Error analysis	10
Reflection	5
Individual learning check	5
Tổng	100

29. Peer assessment dễ chấm

Checklist:

Calculation
☐ Unigram probability đúng
☐ Bigram probability đúng
☐ Sentence probability đúng
☐ Perplexity đúng

Implementation
☐ count n-gram đúng
☐ conditional probability đúng
☐ log probability đúng
☐ smoothing đúng

Experiment
☐ Có train/validation/test
☐ Có so sánh unigram/bigram/trigram
☐ Có giải thích kết quả

Analysis
☐ Có 2 prediction đúng
☐ Có 2 prediction sai
☐ Có giải thích nguyên nhân

30. Mối liên hệ với LAB 01

LAB 01
 Text
 ↓
 Tokenization
 ↓
 Count
 ↓
 TF-IDF
 ↓
 Document representation
 ↓
 Search

sang:

```
LAB 02
Text
↓
Tokenization
↓
Count
↓
Conditional probability
↓
N-gram Language Model
↓
Sentence probability
↓
Perplexity
↓
Next-word prediction
```

Điểm rất hay là **cùng một khái niệm count/frequency nhưng được dùng theo hai hướng khác nhau**:

LAB 01: Count → representation

LAB 02: Count → probability

Từ đây LAB 03 sẽ chuyển sang **Word Representations / Word Embeddings**, và các em sẽ thấy một bước tiến quan trọng:

Sparse count-based representation → Dense distributed representation → Contextual representation

Đó sẽ là cầu nối rất tự nhiên từ NLP cổ điển sang neural NLP và sau này là Transformer/LLM.